COLÉGIO TÉCNICO BENTO QUIRINO
ENSINO MÉDIO TÉCNICO | TÉCNICO EM GAMES — 1º ANO

DISCIPLINA: BANCO DE DADOS

TEMA DA AULA	AMBIENTE PRÁTICO	PRAZO / DURAÇÃO	ENTREGA FINAL
Refinamento e Validação Física	**SQLite Online (sqliteonline.com)**	**2 Aulas (Aulas 32 e 33)**	**Repositório GitHub (.sql)**

1. APRESENTAÇÃO TEÓRICA: POR QUE REFINAR O BANCO DE UM GAME?

No desenvolvimento de jogos, uma falha na estrutura do banco de dados pode causar bugs sérios, como personagens com vida negativa, compras sem saldo de moedas ou itens órfãos que travam o inventário. O **refinamento físico** consiste em adicionar restrições lógicas e matemáticas (*constraints*) diretamente na tabela para que o próprio banco de dados bloqueie erros de execução.

NOT NULL
Garante que atributos essenciais nunca fiquem vazios (ex: o Nick do jogador ou o ID do item).

CHECK (condição)
Cria regras lógicas e matemáticas obrigatórias (ex: CHECK `(vida >= 0)` ou níveis maiores que 0).

DEFAULT valor
Define um valor padrão automático caso nenhum dado seja informado no momento do registro.

FOREIGN KEY
Garante a integridade referencial, impedindo que existam registros dependentes órfãos na memória.

2. DEMONSTRAÇÃO PRÁTICA: ANALISANDO O SCRIPT POR PARTES

Estude os blocos de código abaixo separados por etapa. **Importante:** Não copie e cole o bloco inteiro de uma vez; analise a sintaxe e a função de cada instrução individualmente.

```
-- PASSO 1: Ativação obrigatória de Chaves Estrangeiras no SQLite PRAGMA foreign_keys = ON;
```

Explicação do Passo 1:

- Por padrão, o interpretador do SQLite mantém a verificação de chaves estrangeiras desligada por compatibilidade. O comando `PRAGMA foreign_keys = ON;` deve ser sempre a primeira linha executada no script para ativar a validação referencial.

```
-- PASSO 2: Criação da Tabela Pai (Personagem) com Restrições de Validação CREATE TABLE TB_PERSONAGEM ( ID_Personagem INTEGER PRIMARY KEY AUTOINCREMENT, Nome TEXT NOT NULL UNIQUE, Classe TEXT CHECK (Classe IN ('Guerreiro', 'Mago', 'Arqueiro', 'Ladino')), Nivel INTEGER DEFAULT 1 CHECK (Nivel > 0), Xp_Atual INTEGER DEFAULT 0 CHECK (Xp_Atual >= 0) );
```

Explicação do Passo 2:

- `Nome TEXT NOT NULL UNIQUE`: Impede nomes repetidos no jogo e bloqueia registros sem nome.
- `CHECK (Classe IN (...))`: Aceita apenas as classes declaradas. Qualquer outra entrada será rejeitada pelo banco.
- `Nivel INTEGER DEFAULT 1 CHECK (Nivel > 0)`: Todo personagem começa no nível 1 e não pode ter nível zero ou negativo.

```sql
-- PASSO 3: Criação da Tabela Filho (Missões) Relacionada à Tabela Personagem CREATE TABLE TB_MISSAO ( ID_Missao INTEGER PRIMARY KEY AUTOINCREMENT, Titulo_Missao TEXT NOT NULL, Recompensa_Gold INTEGER DEFAULT 50 CHECK (Recompensa_Gold >= 0), Status_Missao TEXT DEFAULT 'Pendente' CHECK (Status_Missao IN ('Pendente', 'Em Andamento', 'Concluida')), ID_Personagem INTEGER NOT NULL, FOREIGN KEY (ID_Personagem) REFERENCES TB_PERSONAGEM(ID_Personagem) ON DELETE CASCADE );
```

Explicação do Passo 3:

- `Recompensa_Gold ... CHECK (Recompensa_Gold >= 0)`: Garante que o jogo nunca distribua recompensas negativas.
- `FOREIGN KEY (...) REFERENCES ... ON DELETE CASCADE`: Liga a missão ao personagem. Se um personagem for excluído do jogo, suas missões ativas são automaticamente apagadas do banco de dados.

```sql
-- PASSO 4: Teste de Validação (Inclusão Válida vs. Tentativa Inválida) INSERT INTO TB_PERSONAGEM (Nome, Classe) VALUES ('ShadowRider', 'Ladino'); -- Este comando abaixo GERARÁ ERRO proposital pois a classe 'Bardo' não é permitida no CHECK: -- INSERT INTO TB_PERSONAGEM (Nome, Classe) VALUES ('Bardok', 'Bardo');
```

3. ATIVIDADE PRÁTICA DOS ALUNOS (INDIVIDUAL)

⚠️ **REGRA PRINCIPAL DA ATIVIDADE: TEMA LIVRE VOLTADO PARA GAMES!**
Você NÃO utilizará tabelas de aulas anteriores. Cada aluno deve planejar e construir um novo minimodelo relacional do zero contendo **EXATAMENTE DUAS TABELAS** com tema livre sobre jogos digitais (ex: *Inimigos e Drops*, *Lojas e Armas*, *Pokémons e Treinadores*, *Guildas e Membros*, *Partidas e Pontuações*).

Requisitos Obrigatórios para a Modelagem:

1. **Tabela 1 (Entidade Pai):** Criar uma tabela com no mínimo 4 colunas, contendo `PRIMARY KEY`, restrição `NOT NULL` e pelo menos um `CHECK` ou `DEFAULT`.
2. **Tabela 2 (Entidade Filho):** Criar a segunda tabela com no mínimo 4 colunas, contendo `PRIMARY KEY`, `FOREIGN KEY` vinculada à Tabela 1 e pelo menos duas restrições de validação (`CHECK` / `DEFAULT`).
3. **Teste de Carga de Dados (INSERTs):** Escrever comandos de inserção para cadastrar no mínimo 3 registros válidos em cada tabela.
4. **Teste de Bloqueio por Erro:** Escrever um comando `INSERT` com dados propositalmente inválidos (comentado com `--`) demonstrando qual regra de validação iria bloquear a inserção.

4. PASSO A PASSO DE ENVIO PARA O GITHUB

1. Acesse o interpretador online: **https://sqliteonline.com/**
2. No painel do SQLite, digite e teste todas as suas instruções SQL até garantir que não haja erros de sintaxe.
3. Copie o seu script final e salve no seu computador em um arquivo de texto com a extensão `.sql` (Exemplo: `modelagem_game_nome_sobrenome.sql`).
4. Acesse sua conta no **GitHub** (github.com) e crie um novo repositório público (Ex: `refinamento-banco-games`).
5. Faça o upload do arquivo `.sql` clicando em **Add file > Upload files** e confirme em **Commit changes**.
6. Copie o link do seu repositório no GitHub e envie no formulário/tarefa da disciplina.